

# Vulnerability Patching via Structure-Aware Retrieval-Augmented Generation

Master's Thesis

TUM School of Computation,  
Information and Technology

Institut für Informatik  
Lehrstuhl III (I3)  
Prof. Dr. Jana Giceva  
Makreshanska

Boltzmannstr. 3  
Room: 02.11.043  
85748 Garching

<https://www.db.in.tum.de/>

**Supervisor:** Prof. Dr. Jana Giceva Makreshanska  
**Advisor:** Fabian Wenz  
**Email:** [jana.giceva@in.tum.de](mailto:jana.giceva@in.tum.de)  
**Phone:** +49 (0) 89 / 289-17286  
**Starting date:** April 2026

## Context

To meet compliance requirements, organizations must adhere to International Standards and regulations such as ISO/IEC 27001 [?]. This standard establishes best practices for data protection, cyber-resilience, and operational excellence. To accomplish that and improve development robustness, enterprises must anticipate and adapt to vulnerabilities and potential attacks. For software companies, this goal demands evolving software development life cycles to rapidly identify, generate, and deploy risk mitigation for vulnerabilities. One possible approach is to integrate automated systems that proactively generate patches for buggy programs.

With the advent and widespread adoption of Large Language Models (LLMs), a new venue for automated remediation emerged [?]. However, the current LLM approaches remain insufficient for security patching due to two primary failure reasons:

1. **Semantic Misunderstanding:** 84–94% of AI patches rely on localized fixes that fail to address root causes, making them significantly more fragile and easier to bypass than human-authored patches [?].
2. **Vulnerability Identification Failures:** Models frequently misidentify vulnerability contexts, with some retrieval tasks showing failure rates as high as 91.6% [?]. This leads to "semantic hallucinations", where a model might apply mitigation steps for the wrong operating system [?] or package version [?], creating a false sense of security while leaving the actual system exposed.

These failures are aggravated by providing lengthy raw source code to the LLM. In these cases, vulnerability-relevant logic is diluted by implementation details and boilerplate code, thereby degrading the quality of security analysis [?]. Empirical results from the same work suggest that this code noise induces what the authors term Analysis Focus Drift, wherein the model performs a code quality review rather than a vulnerability analysis. Thus, there is an opportunity to develop methodologies that enrich security-relevant information while filtering out implementation noise. Moreover, although various studies show that knowledge bases improve LLM responses in security applications [?, ?, ?, ?], it is unclear which approach optimizes patch quality effectively in multi-agent environments.

This work contributes to the exploration of this gap by proposing a Retrieval-Augmented Generation (RAG) system that uses graph-based representations of code vulnerabilities to generate embeddings.

## Related Work

Automated Program Repair (APR) aims to fix functional and security defects, which are critical in large-scale software systems. With advances in LLM coding capabilities, systems such as RepairAgent [?] have demonstrated that autonomous LLM agents can resolve hundreds of bugs through iterative tool interaction. However, this process comes at a high computational cost, averaging 270,000 tokens per bug, raising concerns about the efficiency of these approaches.

Automated Vulnerability Repair (AVR), a subtask of APR, focuses on addressing software security vulnerabilities while minimizing dependence on domain experts [?]. However, APR techniques do not transfer seamlessly to AVR [?], necessitating specialized approaches. Systems such as FineNib [?] exemplify this complexity by incorporating formal static analysis artifacts, specifically CodeQL queries, as executable verification oracles during code generation. In contrast to approaches that rely on existing security tools, AutoPatch [?] proposes a system grounded through RAG and multiple agent interactions to achieve meaningful results. RAG was originally proposed to combine the flexibility of pre-trained parametric memory with the precision of non-parametric external knowledge [?], enabling models to consult a document index and reason beyond their training data. Despite these advantages, RAG-based approaches identified in this review depend exclusively on textual vector embeddings for retrieval, failing to capture deep structural signatures of vulnerabilities, such as taint flows, data dependency chains, and control flow anomalies. This limitation creates a representational gap, causing these systems to fall back on the inherent capacity of the generative model, assuming that scale is the main lever for repair quality. Consequently, this dependency results in high repair costs and structural logic errors [?].

Prior to the recent advancements in LLM-based code understanding [?], static analysis techniques such as Code Property Graphs (CPGs) [?] were already well established. CPG is a code representation that merges abstract syntax trees, control flow graphs, and data flow graphs into a joint data structure, facilitating vulnerability detection through graph traversal. Although CPGs have been applied to vulnerability detection, their potential as a retrieval representation for automated vulnerability repair remains unexplored.

The proposed approach aims to ground retrieval in graph embeddings, using structural similarity as the primary retrieval signal, rather than relying only on semantic keyword matching.

## Methodology

This work proposes a structure-aware RAG framework for automated vulnerability patching. Rather than relying only on the generative capacity of frontier models, this approach shifts part of the patching complexity to retrieval precision, better grounding the repair process.

The vulnerable code will be represented using graph embeddings to retrieve structurally similar patches from a knowledge base at inference time. Beyond retrieval, we will apply the graph edit distance [?, ?] to identify structural differences between vulnerable and patched code within the retrieved pairs. These structural differences will serve as a grounding repair orientation for the LLM to remediate a flaw.

We will investigate how this design leverages structural meaning encoded in graph representations to identify the lines that introduce security breaches.

## Architecture

The architecture employed in this work is displayed in Figure1, where the box containing  $\Delta$  represents our contribution. All the other components are derived from existing frameworks.

The system consists of a knowledge base (KB) and an agentic framework. The KB is constructed from datasets annotated with CVE (Common Vulnerabilities and Exposures) and CWE (Common Weakness Enumeration). Each entry contains the vulnerable/patched code pair and its structural differences extracted via subgraph matching.

The system will use an agentic framework common in industrial applications and similar to the AutoPatch [?] design and employ two specialized LLM agents:

- **Patcher Agent:** synthesizes code repairs by using the retrieved context in its prompt;
- **Vulnerability Verifier Agent:** receives the generated patch and verifies whether the previously spotted vulnerability persists.

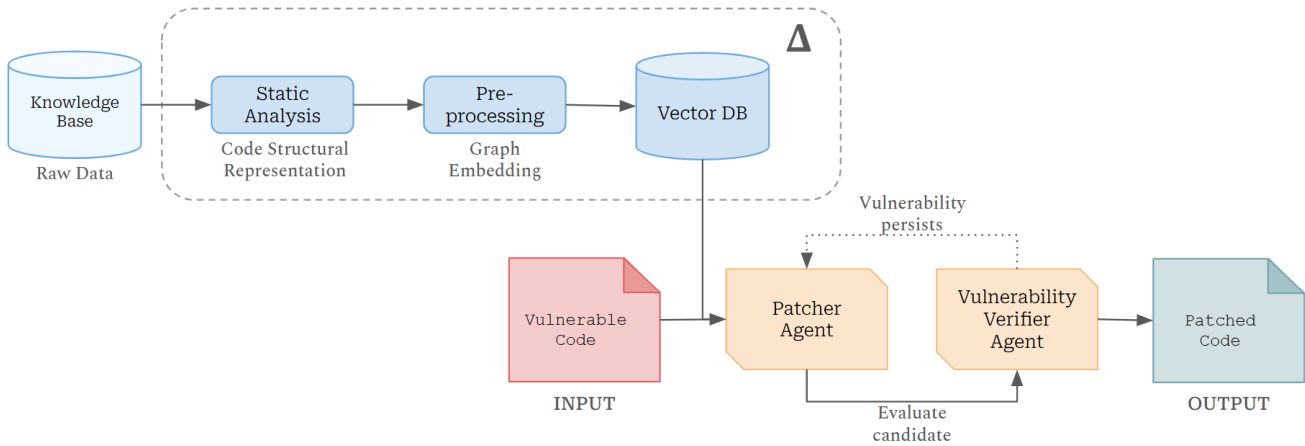


Figure 1: System architecture used in this work to verify the graph embedding usage for patching vulnerabilities, where the  $\Delta$ -box is our contribution.

To evaluate the proposed framework, we address two primary research questions:

**RQ1:** How effective are graph-based embeddings at retrieving relevant vulnerability fix patterns from the knowledge base?

**RQ2:** What are the trade-offs of using Graph-RAG in the vulnerability patching pipeline?

If within reach, we aim to also address the following complementary research questions:

**CRQ1:** How does the structure-aware RAG framework with a small model compare with existing approaches (e.g., zero-shot small models and AutoPatch)?

**CRQ2:** What is the computational and financial cost of the approach (e.g., token usage, storage complexity and size, end-to-end pipeline costs)?

## Data Pipeline

The pipeline begins by generating a graph embedding of the input vulnerable code snippet. This embedding is used to query the KB for the most structurally similar fix. The AutoPatch dataset will be used as the primary source for the KB, providing a consistent baseline for our experiments.

Once the relevant entry is retrieved, the Patcher Agent enriches its prompt with the structural data and other relevant fields to generate a candidate patch. This output is then forwarded to a Vulnerability Verifier Agent for validation.

To manage the indices, the infrastructure requires a graph database or a specialized vector store.

As an ablation study, we would like to evaluate the performance of a small, open-source LLM as the patch generator, comparing its results with and without the KB context enrichment.

## Evaluation

Evaluation will be conducted using a curated dataset of vulnerabilities drawn from the AutoPatch corpus.

- **Retrieval Quality:** Measured via Hit@K, which calculates the fraction of true entities appearing in the top  $k$  results of the ranked list. This metric evaluates the performance of the Graph-based search.

- **Patch vulnerability verification:** Average loop counts will measure the number of iterations required before the vulnerability is no longer detected. Notably important for answering RQ2 and directly comparing results with AutoPatch.
- **Efficiency:** A comparative cost analysis (measuring tokens, as in RepairAgent, and USD, as in AutoPatch) against LLM-only solutions.
- **Patch Quality:** A qualitative assessment by industry security specialists on a representative sample of generated patches.
- **Storage complexity:** We will measure and compare the storage size of a KB using graph embeddings versus one using traditional text embeddings.

## Discussion

## Deliverables

- Source code of the implementation of the RAG-based multi-agent framework.
- The curated benchmark and/or CVEs, vulnerable code samples used for benchmarking, and any further data used to populate the knowledge base